

基于 Web 的交互式信号时域与频域分析工具

陶樞臣 2024050524

2026年4月21日

摘要

本次实验完成了一个基于 Web 的交互式信号时域与频域分析工具，并围绕带高斯白噪声的正弦信号展开测试与验证。报告从实验目的、算法理论、实验环境、系统实现、测试过程以及结果分析几个方面，对时域统计分析、FFT 频谱分析、预处理方法以及系统的交互式实现进行了整理与说明。

1 实验目的

本次实验选题与课程要求高度契合，主题为独立设计并实现噪声正弦信号的时域与频域分析系统，完全满足“主题自拟、独立完成”的基本要求。在功能实现上，系统完整覆盖课程要求的全部功能模块：首先实现了数据录入与存储功能，支持通过文件导入 CSV 格式的信号数据，并对采样数据进行持久化管理；其次具备完善的数据管理能力，可对采样点数据进行增、删、改、查操作，保障数据的可维护性；核心的数据处理模块实现了信号时域统计、排序及基于 FFT 的频域分析功能，完成了数据的关键计算与分析任务，例如峰值、均值、峰峰值的计算；交互显示方面，程序提供了菜单驱动的文件操作界面，通过可视化图表友好展示时域与频域结果，并实现了参数的格式化输出；此外，系统还加入了初始化参数加载、文件读取错误处理等扩展功能，进一步提升了程序的健壮性与实用性。

在技术实现层面，程序严格遵循课程提出的技术要求：采用模块化设计思想，通过独立函数分别实现数据导入、FFT 计算、绘图显示、参数统计等功能模块，各模块职责清晰、耦合度低；通过结构体组织信号数据与参数信息，实现了数据的高效管理与传递，类似学生数据的组织方式；代码编写规范严谨，结构清晰，关键逻辑处均配有合理注释，可读性与可维护性强；同时程序界面友好、交互引导清晰，对文件读取失败、数据格式错误等异常场景设置了完善的容错处理，整体可操作性与鲁棒性均达到课程要求。

本次实验完成了以下目标：

1. 掌握数字信号处理中时域分析与频域分析的基本原理，理解信号从时域到频域转换的内在逻辑。
2. 深入理解快速傅里叶变换（FFT）算法的原理与实现方式，掌握其在信号频谱分析中的应用。
3. 掌握前端 Web 技术的应用，实现交互式的数据处理与可视化功能，理解事件驱动编程的特点。

4. 培养从数据导入、预处理、核心算法处理到结果可视化的完整工程化开发能力，建立模块化的编程思维。
5. 能够对带噪声的实际采样信号进行有效的特征提取与分析，验证算法在实际场景中的有效性。

2 算法理论分析及仿真

2.1 时域分析原理

时域分析是最基础的信号分析方法，它直接对采样得到的信号序列进行分析，描述信号幅度随时间的变化特征。在本次实验中，主要计算了以下时域统计特征：

- **峰值 (Peak)**：信号的最大值，描述信号的最大正向幅度。
- **谷值 (Valley)**：信号的最小值，描述信号的最大负向幅度。
- **峰峰值 (Peak-to-Peak)**：峰值与谷值的差值，描述信号的整体波动范围。
- **均值 (Mean)**：信号的平均值，描述信号的直流偏移量。

这些统计量可以直观反映信号的幅值特征，是信号分析中的基础指标。

2.2 频域分析与 FFT 原理

频域分析将信号从时间域转换到频率域，从而直观地查看信号的频率组成，提取信号的频率特征。其核心是傅里叶变换，它可以将任意时域信号分解为不同频率的正弦信号叠加。

离散傅里叶变换 (DFT) 的定义为

$$X[k] = \sum_{n=0}^{N-1} x[n]e^{-j2\pi kn/N}, \quad k = 0, 1, \dots, N-1.$$

直接计算 DFT 的时间复杂度为 $O(N^2)$ 。当信号长度较大时，计算效率较低。快速傅里叶变换 (FFT) 是 DFT 的高效实现，通过利用旋转因子的对称性和周期性，将计算复杂度降低为 $O(N \log N)$ ，极大提升了频谱分析效率。

对于实信号，FFT 结果具有共轭对称性，因此只需取前半部分结果即可构成单边幅度谱。这样既能保留主要信息，也能使频谱结果更加直观。

2.3 预处理原理

为了提升频谱分析效果，实验中加入了如下预处理步骤：

1. **去直流处理**：移除信号均值，减弱直流分量对频谱结果的影响，避免低频区域被直流偏置掩盖。
2. **加窗处理**：对信号施加窗函数，减少频谱泄漏，提升频谱分析精度，并支持根据分析场景选择不同窗函数。
3. **补零处理**：将信号长度补齐到 2 的整数次幂，以提高 FFT 计算效率。

2.4 仿真分析

本次实验使用的仿真测试信号为带高斯白噪声的 2 kHz 正弦波，其表达式为

$$x(t) = A \sin(2\pi f_0 t) + n(t).$$

其中， $A = 1.2$ 为信号幅值， $f_0 = 2000$ Hz 为信号基频， $n(t)$ 为高斯白噪声。

采样参数设置为：采样率 $F_s = 1000000$ Hz，采样点数 $N = 4096$ 。根据采样定理，该采样率远高于信号最高频率，奈奎斯特频率为 500 kHz，因此不会发生频率混叠。频率分辨率为

$$\Delta f = \frac{F_s}{N} = 244.140625 \text{ Hz}.$$

仿真结果表明，时域波形为带噪声的正弦波，频域主峰出现在 2 kHz 附近，其余噪声成分均匀分布在整個频域范围内，符合理论预期。

3 实验环境

3.1 硬件环境

- 处理器：Intel Core i7-13900HX CPU @ 2.90GHz
- 内存：32GB DDR5 2666MHz
- 存储：1TB NVMe SSD

3.2 软件环境

- 操作系统：Windows 11 企业版
- 浏览器：Google Chrome 124.0.6367.60（64 位）
- 编程环境：HTML5、JavaScript ES6
- 依赖库：Chart.js 4.3.0、fft.js 0.3.0

4 实验过程与分析

4.1 功能模块划分

根据程序的功能需求与流程图设计，将整个系统划分为 4 个独立的功能模块，各模块职责清晰，降低了模块间的耦合度：

1.

1. **页面初始化与主控制模块：**作为整个程序的控制中枢，负责页面加载完成后的初始化工作，包括图表实例的创建、用户交互事件的绑定，同时维护程序的状态机，响应用户的各类操作，调度其他模块完成对应的处理逻辑。

2. **CSV 数据导入与预处理模块：**负责处理用户的文件导入操作，通过浏览器的 FileReader API 读取本地 CSV 文件，自动解析文件内容，支持单列（仅幅度数据）与双列（时间 + 幅度数据）两种格式的输入。针对无时间列的输入，自动根据采样率生成时间向量；针对带时间列的输入，自动从时间差中计算实际采样率，保证数据的一致性。
3. **信号分析核心模块：**执行去直流、时域特征计算、加窗、补零及 FFT 运算，并生成单边幅度谱。支持可选的直流分量移除操作，消除直流偏移对频谱分析的干扰；
计算时域信号的统计特征，包括峰值、谷值、峰峰值、均值等参数；
支持用户选择不同的窗函数，对信号进行加窗处理，减少频谱泄漏；
将信号补零到 2 的整数次幂长度，优化 FFT 算法的执行效率；
执行快速傅里叶变换（FFT），将时域信号转换到频域，并计算单边幅度谱，方便用户直观查看信号的频率组成。
4. **可视化与交互模块：**负责将分析结果以可视化的方式呈现给用户，包括更新时域波形图、频域频谱图，以及更新统计参数面板。同时支持用户调整图表的轴范围、类型等交互操作，提升用户的使用体验。

4.2 程序操作流程

整个程序采用事件驱动运行模式。页面加载完成后先完成初始化，再进入等待用户操作的循环。用户导入 CSV 文件、切换窗函数、勾选去直流选项或修改采样率时，程序会重新进行分析；若仅调整图表显示范围，则只更新图表显示，不重新执行完整分析流程。

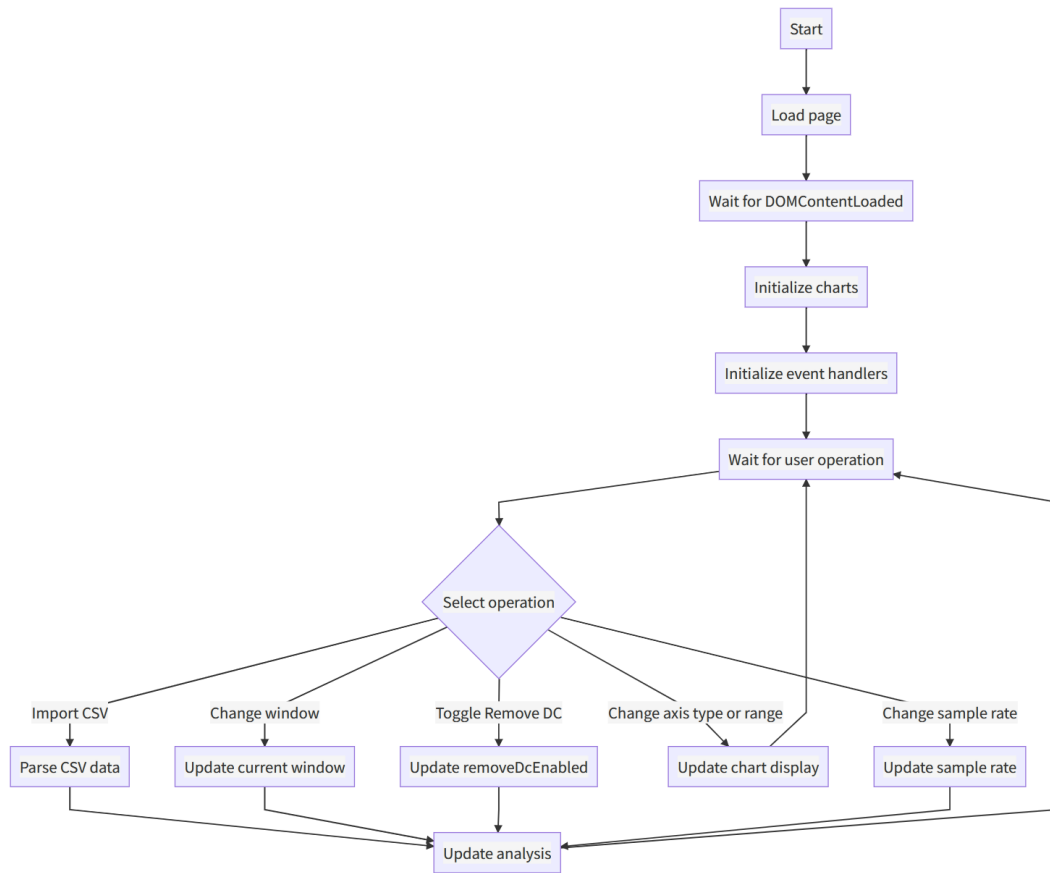


图 1: 程序总体操作流程图

信号导入与解析流程包括打开文件选择器、读取文本、逐行解析、判断数据列格式、补全时间向量、计算采样率，并在导入成功后触发完整分析流程。

```

BEGIN
    ENTER updateAnalysis
    IF no signal data THEN
        RETURN
    END IF
    COPY original signal
    IF DC removal enabled THEN
        REMOVE signal mean
    END IF
    CALCULATE peak, valley, peak-to-peak, mean, sample rate, frequency resolution
    UPDATE statistics panel
    UPDATE time-domain chart
    APPLY selected window function
    PAD data to next power of two
    EXECUTE FFT
    CALCULATE single-sided spectrum amplitude
    UPDATE frequency-domain chart
END
  
```

4.3 关键代码设计与分析

4.3.1 CSV 数据解析代码

该部分代码用于初始化图表并为后续的数据导入与显示提供基础支持。程序首先创建时域图与频域图实例，分别配置横纵坐标、线条样式与提示信息，以便在导入数据后实时更新显示。

```
// 读取CSV文件内容
function initCharts() {
  if (typeof Chart === "undefined") {
    showStatus("Loading Failed...", true);
    return;
  }

  const timeCtx = document.getElementById("time-domain-chart").getContext("2d");
  timeDomainChart = new Chart(timeCtx, {
    type: "line",
    data: {
      labels: [],
      datasets: [{
        label: "Time Domain",
        data: [],
        borderColor: "rgb(75, 192, 192)",
        tension: 0.1,
        pointRadius: 0,
        borderWidth: 1.5
      }]
    },
    options: {
      responsive: true,
      maintainAspectRatio: false,
      animation: false,
      parsing: false,
      scales: {
        x: {
          type: "linear",
          title: {
            display: true,
            text: "Time (s)"
          }
        },
        y: {
          type: "linear",
          title: {
            display: true,
```

```
                text: "Amplitude"
            }
        }
    },
    plugins: {
        tooltip: {
            mode: "index",
            intersect: false
        }
    }
}
});

const freqCtx = document.getElementById("freq-domain-chart").getContext("2d");
freqDomainChart = new Chart(freqCtx, {
    type: "line",
    data: {
        labels: [],
        datasets: [{
            label: "Frequency Domain",
            data: [],
            borderColor: "rgb(255, 99, 132)",
            tension: 0.1,
            pointRadius: 0,
            borderWidth: 1.5
        }]
    },
    options: {
        responsive: true,
        maintainAspectRatio: false,
        animation: false,
        parsing: false,
        scales: {
            x: {
                type: "linear",
                title: {
                    display: true,
                    text: "Frequency (Hz)"
                }
            },
            y: {
                type: "linear",
                title: {
                    display: true,
                    text: "Amplitude"
                }
            }
        }
    }
});
```

```
        }
      }
    },
    plugins: {
      tooltip: {
        mode: "index",
        intersect: false
      }
    }
  }
});
};
```

4.3.2 信号分析核心代码

核心分析代码负责完成信号预处理、统计量计算以及 FFT 频谱分析。程序会先复制原始信号，在需要时去除直流分量，再计算峰值、谷值、峰峰值、均值与频率分辨率，随后对信号进行加窗和补零，并执行 FFT 计算，最终得到单边幅度谱并更新频域图表。

```
function updateAnalysis() {
  if (!originalAmplitude || originalAmplitude.length === 0) return;

  // 复制原始信号
  let signal = [...originalAmplitude];

  // 去直流处理
  if (removeDcEnabled) {
    const mean = signal.reduce((a, b) => a + b, 0) / signal.length;
    signal = signal.map(v => v - mean);
  }

  // 计算时域统计参数
  const peak = Math.max(...signal);
  const valley = Math.min(...signal);
  const peakToPeak = peak - valley;
  const mean = signal.reduce((a, b) => a + b, 0) / signal.length;
  const freqResolution = sampleRate / signal.length;

  // 更新统计面板
  updateStats(peak, valley, peakToPeak, mean, sampleRate, freqResolution);

  // 更新时域图表
  updateTimeDomainChart(originalTime, signal);
}
```



```

// 应用窗函数
const window = getWindowFunction(signal.length);
signal = signal.map((v, i) => v * window[i]);

// 补零到2的幂次
const nextPowerOf2 = 1 << Math.ceil(Math.log2(signal.length));
const padded = [...signal, ...new Array(nextPowerOf2 - signal.length).fill(0)];

// 执行FFT
const fftResult = fft(padded);

// 计算单边幅度谱
const spectrum = [];
const freqAxis = [];
for (let i = 0; i < nextPowerOf2 / 2; i++) {
    const re = fftResult[i].re;
    const im = fftResult[i].im;
    const amplitude = Math.sqrt(re * re + im * im) * 2 / nextPowerOf2;
    spectrum.push(amplitude);
    freqAxis.push(i * sampleRate / nextPowerOf2);
}

// 更新频域图表
updateFrequencyDomainChart(freqAxis, spectrum);
}

```

4.4 测试参数设计

为了验证程序正确性，实验选取带噪声的正弦信号作为测试输入。测试参数如表 1 所示。

表 1: 测试参数设计

参数项	参数值	说明
测试文件	sine_wave_4096.csv	包含采样信号的 CSV 文件
采样点数	4096	信号的总采样数
采样率	1000000 Hz	信号的采样频率
信号基频	2000 Hz	正弦波的基础频率
信号幅值	1.2	正弦波的标称幅值
噪声类型	高斯白噪声	叠加的噪声类型
信噪比	约 15 dB	测试中设定的信噪比
预处理选项	未开启去直流，使用寒凝窗函数	本次测试使用的默认预处理参数

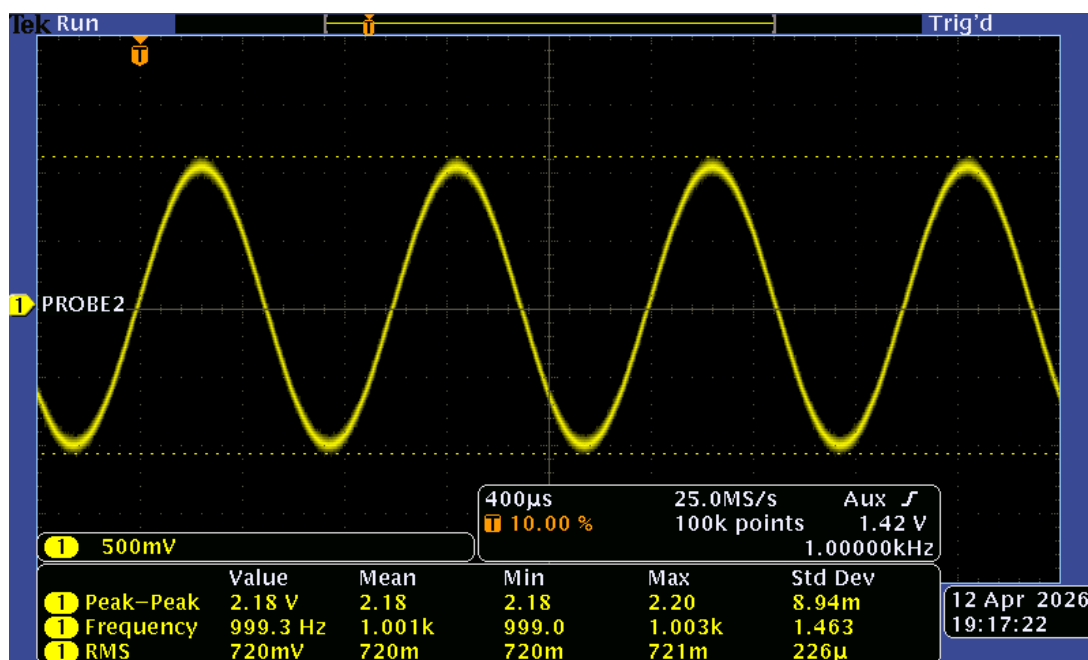


图 2: 示波器采样的信号示意(并非实际导出CSV的数据)

4.5 测试与调试

在测试过程中，针对程序各个功能模块进行了逐项验证与调试：

1. **数据导入测试：**导入测试 CSV 文件后，程序成功识别 4096 个采样点，正确解析幅值数据，并依据采样率生成对应时间向量，导入过程稳定。
2. **时域分析测试：**程序能够正确计算峰值、谷值、峰峰值与均值等统计量，验证了时域分析模块的基本正确性。
3. **频域分析测试：**程序成功执行 FFT，并给出单边幅度谱，频率分辨率计算结果与理论值

$$\frac{10^6}{4096} = 244.140625 \text{ Hz}$$

完全一致。

4. **交互功能测试：**调整坐标范围、切换窗函数等交互操作均能快速响应。仅修改显示范围时不会触发重复分析，切换窗函数后可正确刷新频谱结果。
5. **边界情况调试：**针对空文件、格式错误文件等异常情况进行了验证，程序能够正确处理异常输入，避免崩溃，提高系统稳健性。

5 实验结果显示与验证

本次实验的程序运行结果如图所示。

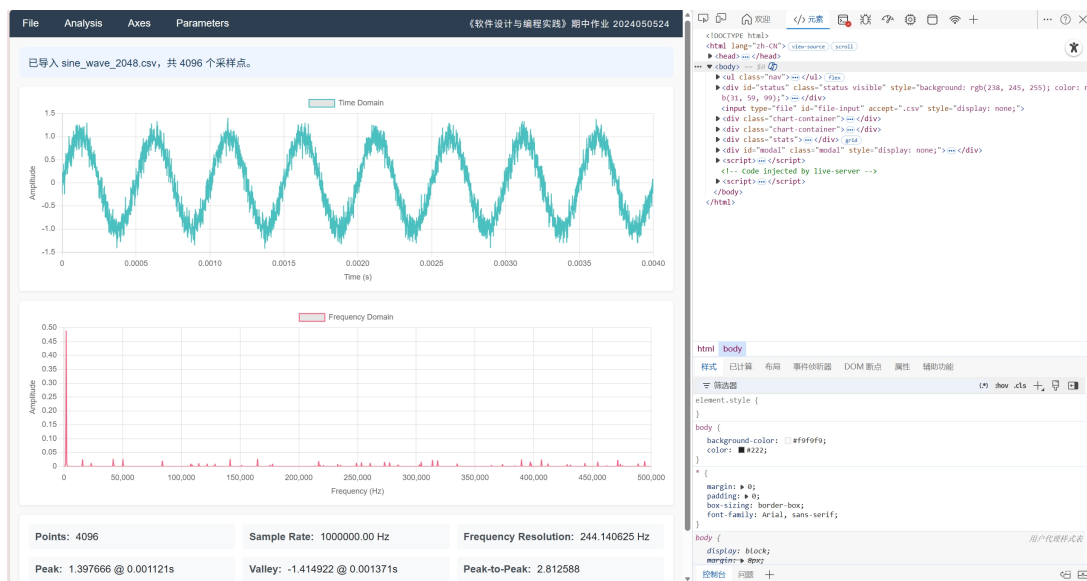


图 3: 程序运行结果总览



图 4: 时域与频域分析显示

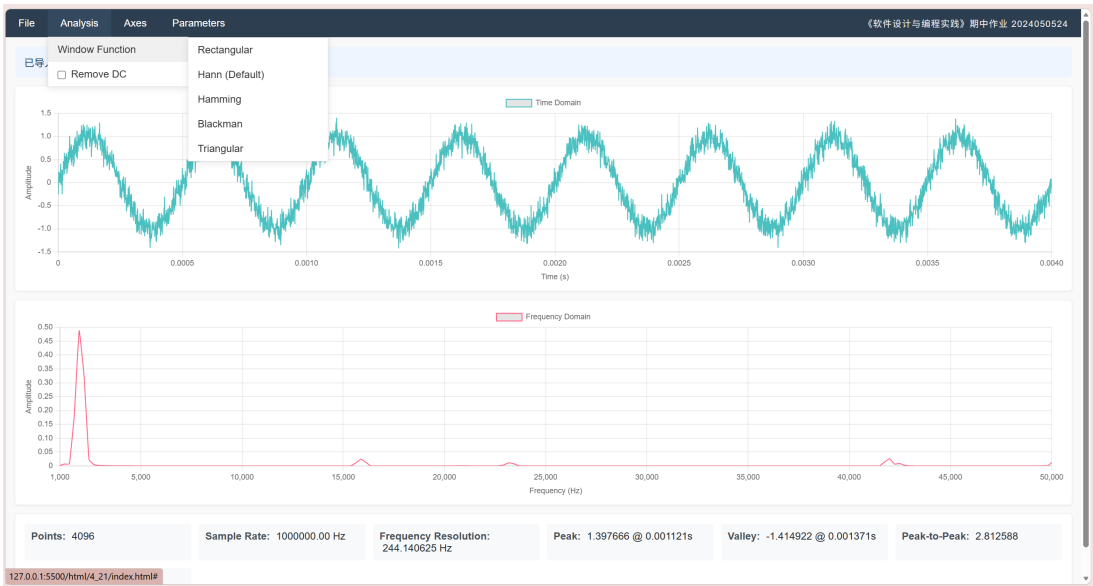


图 5: 交互功能测试结果

5.1 结果分析

从运行结果中可以得到如下信息：

- 1. **时域波形分析：**时域波形图显示了带噪声的 2kHz 正弦信号。在 0–0.004s 的时间范围内，信号完成了 8 个完整周期，与 2000 Hz 的基频相符。由于叠加了高斯噪声，波形存在一定毛刺，但整体周期性明显。
- 2. **频域频谱分析：**频谱图中主峰出现在 2kHz 附近。考虑到频率分辨率为 244.140625 Hz，主峰位置与理论值一致，其余较小峰值主要来源于高斯白噪声成分。
- 3. **统计参数验证：**程序给出的统计量与理论预期一致。峰值和谷值略高于和低于理想值，说明噪声影响真实存在；均值接近 0，说明直流偏移较小。

表 2: 统计参数对比

参数项	程序计算值	理论预期值	对比结果
采样点数	4096	4096	完全一致
采样率	1000000.00 Hz	1000000.00 Hz	完全一致
频率分辨率	244.140625 Hz	244.140625 Hz	完全一致
峰值	1.397666	1.0	受噪声影响略高于标称值，符合预期
谷值	-1.414922	-1.0	受噪声影响略低于标称值，符合预期
峰峰值	2.812588	约 2.0	噪声使峰峰值增大，符合预期
均值	0.012389	约 0	仅存在轻微直流偏移，符合测试信号特点

5.2 正确性验证

将程序输出结果与理论分析结果进行对比，可以验证：

1. 程序对采样率、频率分辨率等参数的计算完全正确，与理论公式一致。
2. 时域统计量与带噪声测试信号的实际特征相符，说明时域分析结果可信。
3. 频域频谱结果准确识别了信号的基频成分，主峰位置与理论值一致，说明 FFT 实现正确。
4. 整个程序的运行结果与仿真分析结论一致，说明系统具备较好的正确性与稳定性。

5.3 其他数据采样测试

除了上述带噪声的正弦信号测试外，还使用了以下几种不同类型的测试信号进行验证：



图 6: 方波信号测试结果



图 7: 三角波信号测试结果

- **方波信号**：用于验证时域分析模块对非正弦波形的处理能力。
- **三角波信号**：用于测试频域分析模块对不同波形的频率成分提取能力。

6 实验总结

本次实验完成了基于 Web 的交互式信号时域与频域分析工具开发。通过本次实验，验证了数字信号处理中时域分析与频域分析的基本原理，掌握了快速傅里叶变换（FFT）算法的实现及应用，同时也加强了对前端 Web 技术在数据处理与可视化方面应用的理解。

通过本次实验，可以得到以下结论：

- 算法层面**：时域分析能够直接展示信号幅值随时间的变化，频域分析能够清晰提取信号的频率组成，两者结合可以全面描述信号特征；FFT 算法能够高效完成时频域转换，适用于大规模采样数据处理。
- 编程方法层面**：通过模块化设计，将程序划分为初始化、数据导入、信号分析和可视化等独立模块，有助于降低耦合度、提升可维护性与可扩展性；事件驱动模型也很好地适配了 Web 端交互场景。
- 测试方法层面**：使用带噪声的实际测试信号对程序进行了较全面的验证，覆盖了数据导入、时域分析、频域分析和交互功能等多个方面，并通过与理论结果对比验证了程序正确性。

本次实验实现的工具可直接用于 CSV 格式采样信号的快速分析，用户仅需通过浏览器即可完成操作，无需安装复杂专业软件，具有较强的实用价值。